

Angular Micro-Frontend — Interview Cheatsheet

Module Federation • Host/Remote setup • Shared deps • Communication • Routing • Versioning

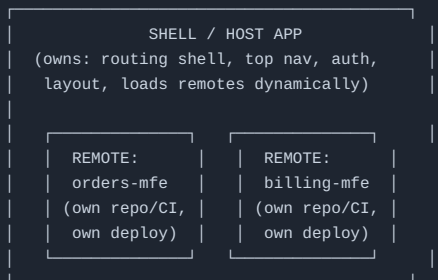
1. What & Why (Definitions)

Micro-frontend (MFE): Splitting a large frontend into independently developed, built, and deployed applications, composed together in the browser (or at build time) into one experience.

Module Federation (MF): A Webpack 5 (and now [@angular-architects/native-federation](#) / esbuild-based) feature that lets one app **dynamically load JS from another app's build output at runtime** — no shared build step needed.

Interview line: "MFEs let independent teams own end-to-end vertical slices — build, test, deploy on their own timeline — while still composing into one product at runtime."

2. Host / Remote Architecture



Role	Responsibility
Host (Shell)	Bootstraps the app, owns top-level routing, layout, auth/session, and dynamically loads remote entry points at runtime.
Remote	Independently built/deployed app exposing specific modules/components via <code>exposes</code> config — consumed by host or other remotes.
Bidirectional host	An app that is both a host (loads others) AND a remote (exposes itself) — common in "MFE mesh" setups.

3. Two Implementation Approaches

Approach	Notes
Webpack Module Federation	Original approach via @angular-architects/module-federation . Requires Webpack-based builder (not esbuild), config in <code>webpack.config.js</code> .
Native Federation	Newer, esbuild/Vite-compatible via @angular-architects/native-federation . Works with Angular's default esbuild builder — recommended for new projects since Angular 17+ defaults to esbuild.

Q: Why did Native Federation emerge?

Angular CLI moved to esbuild for faster builds; classic Webpack Module Federation needs Webpack directly, creating friction. Native Federation replicates MF's runtime behavior using import maps + esbuild, keeping the esbuild build pipeline intact.

4. Core Config — Remote (exposes)

```
// federation.config.js (remote: orders-mfe)
const { withNativeFederation, shareAll } = require('@angular-architects/native-federation/config');

module.exports = withNativeFederation({
  name: 'orders-mfe',
  exposes: {
    './OrdersModule': './src/app/orders/orders.module.ts',
    './OrderListComponent': './src/app/orders/order-list.component.ts',
  },
  shared: { ...shareAll({ singleton: true, strictVersion: true, requiredVersion: 'auto' }) },
});
```

5. Core Config — Host (remotes)

```
// federation.config.js (host/shell)
module.exports = withNativeFederation({
```

```
shared: { ...shareAll({ singleton: true, strictVersion: true, requiredVersion: 'auto' }) },
});
```

```
// federation.manifest.json (host) — maps remote name → remoteEntry URL
{
  "orders-mfe": "http://localhost:4201/remoteEntry.json",
  "billing-mfe": "http://localhost:4202/remoteEntry.json"
}
```

6. Dynamic Loading at Runtime (Host Side)

```
// app.routes.ts (host)
import { loadRemoteModule } from '@angular-architects/native-federation';

export const routes: Routes = [
  {
    path: 'orders',
    loadChildren: () =>
      loadRemoteModule({
        remoteName: 'orders-mfe',
        exposedModule: './OrdersModule',
      }).then(m => m.OrdersModule),
  },
];
```

Key idea: The remote isn't bundled into the host's build. It's fetched over the network **at runtime** via its `remoteEntry.json` — enabling independent deploys without rebuilding the host.

7. Shared Dependencies (Avoiding Duplicate Angular/RxJS)

Option	Meaning
<code>singleton: true</code>	Only ONE copy of this library loaded across host + all remotes (critical for Angular core, RxJS, NgRx — multiple copies break DI/change detection).
<code>strictVersion: true</code>	Throws a runtime error if versions mismatch, instead of silently loading a second copy.
<code>requiredVersion: 'auto'</code>	Automatically reads required version from each project's own <code>package.json</code> .
<code>eager: true/false</code>	Eager = bundled upfront (bigger initial load); lazy (default) = fetched on demand.

Trap Q: What breaks if Angular core isn't shared as singleton?

Each remote loads its own Angular instance → change detection, DI, zone.js all become disconnected between host and remote — components render but events, forms, and services stop working correctly across boundaries.

8. Communication Between Micro-Frontends

Mechanism	When to use
Custom Events / EventBus (<code>window</code> events, RxJS Subject in a shared lib)	Most common — loosely coupled pub/sub, no direct dependency between remotes.
Shared state library (NgRx / signal store in a shared singleton lib)	When remotes need synchronized state (e.g. cart count, auth/user session) — must be marked <code>singleton</code> .
Input/Output via wrapper component	Host wraps a loaded remote component and passes Inputs/listens Outputs like any Angular component.
URL / route params	Simplest — pass context (ids, filters) through the URL; remotes read via <code>ActivatedRoute</code> .
Custom Elements (Web Components)	Framework-agnostic MFEs (e.g. React remote inside Angular host) — expose remote as a custom element, communicate via DOM attributes/events.

Interview line: "I'd default to a shared event bus for loose coupling, and reach for a shared singleton store only when multiple MFEs genuinely need synchronized state like auth or cart."

9. Routing Across Micro-Frontends

- Shell owns the top-level `Routes` array; each remote's routes are lazy-loaded under a path prefix (`/orders/**`).
- Remotes can also be run **standalone** (own dev server, own routing) for independent development — same routes work whether loaded standalone or federated into the shell.
- Deep-linking works normally since the shell's router owns the URL; the remote's internal router handles sub-paths once loaded.

10. Versioning & Independent Deployment

Concern	How it's handled
Remote deploys independently	Host only needs the remote's URL (from manifest) — as long as the exposed module's public contract (Inputs/Outputs/API) doesn't break, host doesn't need a rebuild.
Breaking contract changes	Version the exposed module/route (e.g. <code>/v2/OrdersModule</code>) or coordinate a contract-testing step in CI so host and remote don't drift silently.
Environment-specific remote URLs	Manifest file per environment (dev/staging/prod) — swapped at deploy time, not hardcoded in code.
Rollback	Since remotes deploy independently, rolling back one remote's deployed bundle doesn't require redeploying the host or other remotes.

11. Testing & CI Considerations

- Each remote has its own unit/e2e test suite + pipeline — independent of host's pipeline.
- Integration/contract testing: verify exposed module's shape (Inputs/Outputs, routes) matches what host expects — catches breaking changes before deploy.
- E2E across the composed app (host + real remotes) still needed periodically to catch runtime integration issues that unit tests miss.

12. Micro-Frontend vs Nx Monorepo — Don't Confuse Them

	Nx Monorepo (libs)	Micro-Frontend (Module Federation)
Deployment	Single build, single deploy artifact per app	Multiple independent build + deploy artifacts
Code sharing	Compile-time (imports resolved at build)	Runtime (remote fetched over network)
Team independence	Same repo, same release cadence typically	Can be fully independent repos/pipelines
Can combine both?	Yes — Nx has first-class Module Federation generators (<code>@nx/angular:host</code> , <code>@nx/angular:remote</code>) — monorepo structure + runtime MFE composition together.	

Likely follow-up: "Why use MF inside an Nx monorepo instead of just Nx libs?" → When teams need **independent deployment**, not just independent code organization — libs still ship as part of one build; MF remotes ship separately.

13. Rapid-Fire Interview Questions

Question	Key Answer
What problem does MF solve that Nx libs don't?	Independent deployability — libs still compile into one artifact; MF remotes deploy separately, at their own cadence.
What happens if <code>remoteEntry.json</code> is unreachable?	Loading fails at runtime — must handle gracefully (fallback UI, retry, error boundary) since it's a network dependency, not a build-time guarantee.
How do you avoid version conflicts for Angular/RxJS?	<code>shareAll + singleton: true + strictVersion: true</code> , keep versions aligned across host/remotes via shared <code>package.json</code> or renovate bot.
Can a remote be developed/run standalone?	Yes — each remote is a full Angular app with its own dev server; federation config only matters when composed into the host.
How to share a design system / UI kit across MFEs?	Publish as an npm package (or shared federation module marked singleton) so styling/components stay consistent without duplicating bundle size.
Performance concern with many remotes?	Each remote lazy-loads its own vendor chunk unless shared — audit shared deps carefully to avoid duplicate Angular/RxJS/zone.js bundles bloating load time.

14. Useful Commands

Command	Use case
<code>ng add @angular-architects/native-federation</code>	Add Native Federation to an existing Angular app
<code>ng g @angular-architects/native-federation:init --port 4201 --type remote</code>	Configure app as a remote
<code>ng g @angular-architects/native-federation:init --port 4200 --type dynamic-host</code>	Configure app as a dynamic host
<code>nx g @nx/angular:host shell --remotes=orders,billing</code>	(Nx) Generate a host app pre-wired to given remotes
<code>nx g @nx/angular:remote orders --host=shell</code>	(Nx) Generate a remote app registered with a host

```
nx run-many --target=serve --  
projects=shell,orders,billing
```

(Nx) Serve host + remotes together locally

Prepared for Angular Micro-Frontend / Module Federation interview revision — covers architecture, config, communication, versioning, and Nx integration.